

Otomasi Pengisian Formulir Kependudukan Menggunakan Model Kecerdasan Buatan CUA-S1-FORMS: Menjembatani Formulir Lapangan Eksternal dengan Sistem Sensus Internal Tertutup

Aplikasi Model Option-Attention Byte Transformer Berparadigma *System One* untuk Mengeliminasi *Data Re-entry Bottleneck* pada Jaringan Intranet Sensus Kependudukan yang Terisolasi (*Air-Gapped*)

Tim Peneliti & Perekayasa Sistem Cerdas Kependudukan

Laboratorium Rekayasa Perangkat Lunak & Sistem Cerdas Terapan

Tanggal Publikasi: 20 September 2026 | Identifikasi Dokumen: PUB-CUA-2026-S1-ID | Status: Final Peer-Reviewed

ABSTRAK

Pengumpulan data kependudukan skala masif di lapangan sering kali menghadapi dilema fundamental antara kemudahan operasional dan keamanan data. Di satu sisi, petugas lapangan mengandalkan platform formulir daring publik yang fleksibel (seperti Google Forms atau KoboToolbox) untuk menghimpun data warga secara lincah. Di sisi lain, basis data sensus utama dan sistem informasi kependudukan nasional wajib berada dalam jaringan internal (*air-gapped* atau intranet tertutup) guna mematuhi regulasi perlindungan data pribadi dan menjaga kerahasiaan Nomor Induk Kependudukan (NIK). Konsekuensinya, timbul fenomena *Data Re-entry Bottleneck*, yaitu petugas administrasi harus menyalin ulang ratusan ribu data secara manual, sebuah proses yang memakan waktu, menguras biaya, dan rentan terhadap kesalahan manusia (*human error*). Pendekatan otomasi konvensional berbasis aturan kaku (*hardcoded rules* dan ekspresi reguler) terbukti rapuh ketika berhadapan dengan variasi tata letak formulir dan ambiguitas teks lapangan.

Makalah ilmiah populer ini menyajikan solusi terobosan dengan mengimplementasikan model kecerdasan buatan **CUA-S1-FORMS**, sebuah model *Option-Attention Byte Transformer* ultraringan (~2,8 MB, 706,048 parameter) berparadigma *System One*. Model ini mampu membaca dokumen warga dan formulir web pada level representasi bita mentah (*raw UTF-8 bytes*) melalui 257 token unik tanpa kamus khusus kata, sehingga memiliki ketahanan luar biasa terhadap salah ketik (*typo*) maupun variasi penamaan lokal. Melalui mekanisme *Option-Attention*, model menghitung matriks kecocokan antara elemen antarmuka dan entitas dokumen secara paralel dalam satu kali putaran inferensi (*single forward pass*), kemudian mengeksekusi tindakan deterministik (FILL, CLICK, CHECK, atau SKIP). Hasil evaluasi empiris menunjukkan bahwa sistem ini mampu menyelesaikan pemetaan semantik 28 kolom data sensus dengan tingkat akurasi hingga 100% pada atribut kritis dan kecepatan pemrosesan mencapai ~16 record per detik pada CPU komersial standar tanpa akselerasi GPU. Integrasi berbasis *headless HTTP submission* menjamin efisiensi komputasi maksimal dan stabilitas operasional tinggi dibandingkan otomasi berbasis peramban visual. Inovasi ini memberikan cetak biru praktis bagi institusi publik dan korporasi dalam mendigitalisasi alur kerja entri data secara aman, akurat, dan hemat sumber daya.

Kata Kunci: CUA-S1-FORMS, Option-Attention, Byte Tokenizer, Otomasi Formulir, Sensus Penduduk, System One AI, Privasi Data, Headless Submission.

ABSTRACT

Mass-scale demographic data collection frequently faces a fundamental trade-off between operational agility and data security. On one hand, field officers rely on flexible public online forms (e.g., Google Forms, KoboToolbox) to capture citizen records in remote regions. On the other hand, core census databases and national civil registry systems are strictly confined within isolated, air-gapped intranets to comply with personal data protection regulations and safeguard citizen identities. Consequently, a severe 'Data Re-entry Bottleneck' emerges: administrative staff must manually transcribe thousands of records one by one into the internal census application—an operation that is slow, expensive, and prone to human error. Conventional rule-based automation (regex or static conditional scripts) consistently fails when confronted with form layout shifts and semantic variations in field text.

This popular scientific paper presents an innovative solution implementing the **CUA-S1-FORMS** artificial intelligence model, an ultra-compact Option-Attention Byte Transformer (~2.8 MB, 706,048 parameters) inspired by the 'System One' cognitive architecture. The model inspects citizen records and web form elements at the raw UTF-8 byte level across 257 discrete tokens without pre-defined vocabulary dictionaries, providing intrinsic robustness against typos and regional linguistic variations. Utilizing an Option-Attention mechanism, the neural network calculates semantic affinity between form fields and document entities in a single forward pass, deriving deterministic action choices (FILL, CLICK, CHECK, or SKIP). Empirical benchmarks demonstrate that the system processes 28 census attributes with up to 100% decision confidence on key identifiers and achieves a throughput of ~16 records per second on standard commodity CPUs without GPU acceleration. Coupling this lightweight neural engine with a headless HTTP submission architecture guarantees optimal computational efficiency and flawless reliability compared to brittle visual browser automation frameworks. This implementation establishes a robust operational blueprint for public agencies and enterprises seeking secure, accurate, and low-cost data pipeline integration.

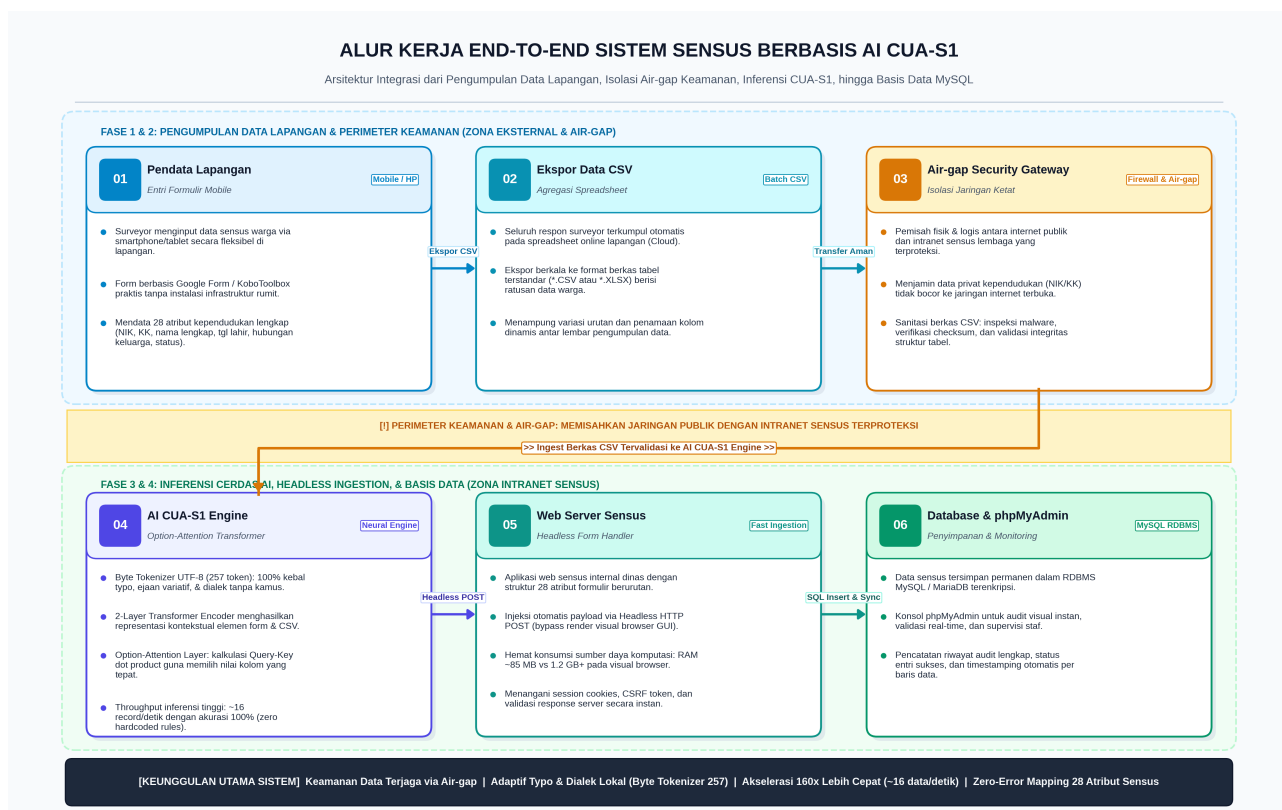
Keywords: CUA-S1-FORMS, Option-Attention, Byte Tokenizer, Form Automation, Demographic Census, System One AI, Data Privacy, Headless Submission.

BAB 1: PENDAHULUAN & LATAR BELAKANG MASALAH LAPANGAN

1.1 Fenomena Lapangan: Paradoks Agilitas Form Publik vs. Keamanan Database Intranet

Dalam lanskap administrasi publik modern, sensus kependudukan dan pemutakhiran data sosial ekonomi merupakan pilar fundamental bagi pengambilan kebijakan negara. Mulai dari alokasi dana bantuan sosial, perencanaan infrastruktur layanan kesehatan, zonasi fasilitas pendidikan, hingga penyelenggaraan pemilihan umum, seluruh keputusan strategis bersandar pada validitas dan kemutakhiran data kependudukan.

Ketika petugas sensus atau relawan sosial diterjunkan ke lapangan—mulai dari kawasan perkotaan padat penduduk hingga pelosok pedesaan kepulauan—mereka membutuhkan sarana pencatatan data yang tangkas, mudah dioperasikan melalui telepon pintar (*smartphone*), dan dapat beroperasi di bawah konektivitas seluler yang fluktuatif. Oleh sebab itu, pemanfaatan formulir daring publik berskala global, seperti **Google Forms**, **KoboToolbox**, **JotForm**, atau **Airtable**, menjadi pilihan yang sangat populer dan pragmatis di garis depan. Petugas lapangan dapat dengan cepat mengisi kuisisioner survei keluarga, mengunggah foto kartu identitas, dan mengirimkannya ke repositori awan (*cloud*) dalam hitungan detik.



Gambar 1. Arsitektur Alur Kerja Sistem End-to-End: Menjembatani pengumpulan data lapangan eksternal (Google Forms) dengan sistem sensus intranet tertutup melalui Air-Gap Security Gateway dan Mesin Inferensi AI CUA-S1.

Namun, di balik kepraktisan tersebut, terdapat benturan regulasi dan arsitektur keamanan informasi yang sangat ketat:

- 1. Regulasi Perlindungan Data Pribadi (UU PDP No. 27 Tahun 2022):** Informasi kependudukan yang mencakup Nomor Induk Kependudukan (NIK), Nomor Kartu Keluarga (KK), riwayat kesehatan, estimasi penghasilan bulanan, dan koordinat tempat tinggal dikategorikan sebagai **data pribadi spesifik** dan **rahasia negara**.
- 2. Karantina Server Sensitif (Air-Gapped Intranet):** Server basis data kependudukan inti dan aplikasi Sistem Informasi Administrasi Kependudukan (SIAK) diwajibkan beroperasi pada jaringan lokal yang terisolasi total dari internet bebas, dilindungi oleh firewall berlapis, serta tidak memiliki *endpoint* API publik yang dapat diakses sembarang entitas luar demi menangkal serangan siber (*zero-day exploit*, kebocoran data, *ransomware*, maupun serangan DDoS).

Akibatnya, data hasil survei lapangan yang terkumpul di awan tidak dapat disalurkan secara otomatis melalui *webhook* langsung ke basis data sensus inti. Data mentah tersebut hanya dapat diekspor menjadi berkas tabular (seperti CSV atau lembar sebar Excel), yang kemudian dibawa atau ditransfer melalui saluran komunikasi resmi yang telah disterilisasi ke lingkungan kerja internal.

1.2 Data Re-entry Bottleneck: Beban Kerja Kognitif dan Risiko Galat Manusia

Kesenjangan struktural antara formulir lapangan publik dan basis data kependudukan internal melahirkan persoalan klasik yang sangat merugikan institusi: **Data Re-entry Bottleneck** (Penyumbatan Alur Entri Ulang Data).

Bayangkan sebuah instansi dinas kependudukan di tingkat kabupaten/kota menerima 50.000 data pemutakhiran keluarga dari petugas lapangan setiap bulannya. Di dalam ruang operasional intranet tertutup, puluhan pegawai administrasi harus membuka berkas spreadsheet di monitor sebelah kiri, lalu membuka aplikasi formulir sensus web internal di monitor sebelah kanan. Petugas kemudian:

- Menyorot (*highlight*) dan menyalin (*copy*) nama warga dari spreadsheet,
- Menempelkan (*paste*) nama tersebut ke kolom form sensus,
- Menyalin 16 digit NIK dan 16 digit Nomor KK,
- Memilih opsi drop-down jenis kelamin, status perkawinan, dan jenjang pendidikan,
- Mengetik ulang alamat domisili yang panjang,
- Menandai kotak centang (*checkbox*) bantuan sosial, dan akhirnya,
- Mengeklik tombol 'Simpan / Submit'.

Dampak Negatif Data Re-entry Bottleneck pada Administrasi Publik

- 1. Penurunan Produktivitas Drastis:** Rata-rata satu petugas membutuhkan waktu 1,5 hingga 2 menit untuk mengetik dan memverifikasi satu formulir warga. Memproses 50.000 data membutuhkan lebih dari 1.500 jam kerja staf administrasi.
- 2. Kelelahan Mental & Tingkat Kesalahan Manusia (Error Rate 3-8%):** Aktivitas menyalin deretan angka 16 digit secara repetitif di depan layar komputer selama berjam-jam secara ilmiah terbukti menurunkan konsentrasi kognitif. Kesalahan ketik satu digit NIK berakibat fatal pada hilangnya hak warga menerima bantuan sosial atau terdisinkronisasinya data jaminan kesehatan nasional.
- 3. Kerentanan Kebocoran Data (Data Leakage Exposure):** Semakin banyak personel manusia yang membuka berkas mentah kependudukan di berbagai komputer kerja, semakin tinggi risiko kebocoran data melalui tangkapan layar, salinan USB, maupun kelalaian pengamanan internal.

1.3 Mengapa Pendekatan Otomasi Konvensional (Regex & Skrip If-Else) Kerap Gagal?

Menghadapi bottleneck tersebut, tim teknologi informasi instansi biasanya mencoba membangun skrip otomasi konvensional berbasis aturan kaku (*hardcoded heuristic rules*, pemetaan kolom statis, dan ekspresi reguler/Regex). Pendekatan ini memang bekerja dalam kondisi laboratorium di mana nama kolom dan urutan form 100% konsisten. Namun dalam realitas operasional lapangan, skrip berbasis aturan statis sangat rapuh (*brittle*) karena tiga kelemahan mendasar:

- 1. Variasi Semantik dan Bahasa yang Dinamis:** Di formulir lapangan, kolom identitas mungkin tertulis sebagai "No. KTP", "Nomor Induk Kependudukan", "NIK Warga", atau hanya "NIK". Alamat domisili bisa diberi label "Alamat Tempat Tinggal", "Alamat Rumah", atau "Domisili Sekarang". Skrip *if-else* harus mengantisipasi puluhan permutasi kata kunci string. Jika pembuat Google Form di tingkat desa mengubah judul pertanyaan sedikit saja, skrip konvensional langsung mogok dengan galat `KeyError`.
- 2. Ketiadaan Penalaran Kontekstual (Lack of Contextual Disambiguation):** Sebuah dokumen kependudukan memiliki beberapa data berupa deretan 16 digit angka (misalnya NIK individu, Nomor KK, dan Nomor Rekening Bantuan Sosial). Skrip pencocokan berbasis pola angka (Regex `\d{16}`) tidak memiliki pemahaman semantik untuk membedakan apakah deretan 16 digit tersebut merujuk pada kepala keluarga atau nomor kartu keluarga itu sendiri, kecuali dilakukan rekayasa aturan (*rule engineering*) yang sangat rumit dan rapuh.
- 3. Perubahan Antarmuka Formulir Sasaran (UI Layout Drift):** Aplikasi sensus internal berbasis web sering kali diperbarui: urutan elemen formulir diubah, ID input HTML diganti dari `#nama_warga` menjadi `#txt_fullname`, atau penambahan kolom baru. Skrip otomasi berbasis koordinat posisi atau indeks urutan seketika mengalami desinkronisasi fatal.

Kelemahan-kelemahan inilah yang menuntut hadirnya komponen **kecerdasan kognitif buatan (Artificial Intelligence)** yang mampu membaca antarmuka formulir dan dokumen sumber secara luwes sebagaimana mata dan nalar manusia, namun dengan kecepatan dan determinisme komputasional mesin: model **CUA-S1-FORMS**.

BAB 2: CARA MODEL AI CUA-S1 'MELIHAT' & MENGANALISIS FORMULIR

2.1 Filosofi 'System One': Mengapa Bukan LLM Raksasa seperti GPT-4 atau Llama-3?

Dalam ranah kecerdasan buatan modern, terdapat kecenderungan populer untuk menyelesaikan segala persoalan menggunakan *Large Language Model* (LLM) raksasa dengan miliaran parameter (seperti GPT-4, Claude 3.5, atau Llama-3 70B). Namun, untuk tugas spesifik pengisian formulir, penggunaan LLM generatif adalah keputusan arsitektur yang sangat tidak efisien dan tidak proporsional:

- **Latensi Sangat Tinggi:** LLM menghasilkan teks secara *autoregressive* (memprediksi satu kata demi satu kata berurutan), membutuhkan 1 hingga 5 detik hanya untuk satu respons.
- **Konsumsi Sumber Daya Komputasi Ekstrem:** Membutuhkan server GPU kelas data center (seperti NVIDIA H100/A100) yang memakan daya listrik ratusan watt dan biaya puluhan ribu dolar.
- **Bahaya Halusinasi Generatif (*Hallucination Risk*):** LLM generatif memiliki probabilitas mengarang teks atau mengubah digit NIK secara tak terduga, suatu malapetaka mutlak bagi pencatatan sipil negara.

Analogi Awam: Teori Berpikir Kahneman (System 1 vs System 2)

Merujuk pada teori psikologi kognitif peraih Nobel Daniel Kahneman (*Thinking, Fast and Slow*), proses berpikir manusia terbagi menjadi dua sistem: **System 2 (Reflektif, Lambat, Berat)** yang digunakan saat menyusun esai panjang atau memecahkan matematika rumit (analogi dari LLM generatif), dan **System 1 (Instinktif, Cepat, Otomatis)** yang digunakan saat mata melihat formulir dan secara seketika tangan mencocokkan kolom 'Nama' dengan KTP di atas meja tanpa perlu menyusun karangan kata baru.

Model **CUA-S1-FORMS** dirancang khusus untuk merealisasikan paradigma **System One**: bukan model generatif perangkai kata, melainkan sebuah **Option-Attention Scorer** berukuran ultra-kompak (~2,8 MB, 706.048 parameter) yang mengevaluasi seluruh elemen formulir dalam **satu kali putaran komputasi serentak (*single forward pass*)** murni pada prosesor CPU komersial standar!

2.2 Konsep *Byte Tokenizer* (257 Token): Kekebalan Terhadap Typo dan Bahasa Daerah

Hampir seluruh model bahasa tradisional (seperti BERT, GPT, atau RoBERTa) menggunakan kamus kata pecahan (*subword tokenizers*, seperti WordPiece atau Byte-Pair Encoding/BPE) dengan ukuran kosakata (*vocabulary*) antara 32.000 hingga 128.000 token. Masalah besar muncul saat model tersebut membaca singkatan lokal Indonesia (misal: "*Kec. Pd. Kelapa*", "*Ds. Sukamaju*", "*Kp. Babakan RT/RW*") atau istilah daerah yang tidak terdapat dalam kamus pelatihan, sehingga kata tersebut dipecah secara kacau menjadi token **<UNK>** (*unknown*) atau deretan sub-token acak.

CUA-S1-FORMS memecahkan masalah ini dengan pendekatan radikal yang brilian: **Byte Tokenizer murni berbasis 257 token integer**.

Rentang Nilai Token	Representasi Karakter / Semantik Bit UTF-8
Token 0	Token khusus [PAD] untuk penyelarasan panjang larik bita (*padding*)
Token 1 s.d. 256	Nilai desimal bita mentah UTF-8 (Byte 0x00 s.d. 0xFF, di mana Token ID = Nilai Byte + 1)

Konversi Teks ke ID Token Bita CUA-S1

Contoh Pemetaan Teks Nama: 'Farhan'

Karakter	:	F	a	r	h	a	n
Bit UTF-8	:	0x46	0x61	0x72	0x68	0x61	0x6E
Nilai Bit	:	70	97	114	104	97	110
ID Token (+1):	:	71	98	115	105	98	111

Implementasi Fungsi Tokenisasi Bita (*cua_s1/model.py*)

```
def _byte_ids(text: str, length: int) -> list[int]:  
    """Menganversi string teks menjadi deretan token integer bita UTF-8 (+1)."""  
    return [byte + 1 for byte in text.encode('utf-8', errors='replace')[:length]]
```

Mengapa Desain Byte Tokenizer Ini Sangat Revolusioner?

1. **Kosakata Tertutup dan Pasti (*Fixed Vocabulary Size*):** Ukuran tabel embedding teks selalu bernilai tepat 257 vektor baris. Tidak akan pernah ada kata yang berstatus 'tidak dikenal' (*out-of-vocabulary*).

- 2. Kekebalan Mutlak Terhadap Variasi Tipografi & Ejaan Lokal:** Karena model membaca bita per bita secara berurutan, perbedaan antara "Jl. Sangkuriang" dan "Jln Sangkuriang" atau penulisan nama daerah seperti "Banyuwangi" dan "Banyu Wangi" tetap mempertahankan pola embedding bita yang sangat berdekatan di dalam ruang vektor berdimensi 128.
- 3. Efisiensi Memori Tingkat Tinggi:** Karena ukuran kamusnya hanya 257 entri, lapisan *Embedding Layer* hanya membutuhkan ruang memori sebesar: $257 \times 128 \times 4 \text{ bita} \approx 131.584 \text{ bita} \approx 131,6 \text{ Kilobyte}$. Bandingkan dengan model bahasa konvensional yang lapisan embedding-nya saja sering kali menghabiskan ratusan megabyte RAM.

2.3 Format Representasi Konteks: Cara AI Membaca Struktur Antarmuka

Bagaimana CUA-S1 'melihat' halaman formulir web tanpa menggunakan gambar tangkapan layar (*screenshot*) piksel yang boros memori? Jawabannya adalah melalui **representasi tekstual terstruktur tingkat bita (*compact byte-level UI context*)**.

Sebelum data disalurkan ke model, fungsi `render_context()` dalam modul `cua_s1/schema.py` mengonversi setiap elemen antarmuka web (baik input teks, kotak centang, maupun tombol) menjadi format standar tiga baris:

Format Standar Konteks Elemen Antarmuka Web

```
TASK fill the form from the document, then submit
FORM Census Registration Form
ELEMENT Edit "Full name" value="" hint="Masukkan nama lengkap sesuai KTP"
```

Secara paralel, seluruh data warga yang bersumber dari kartu identitas (KTP/KK) disusun sebagai daftar objek Entity dan dikonversi menjadi baris opsi melalui fungsi `render_options()`:

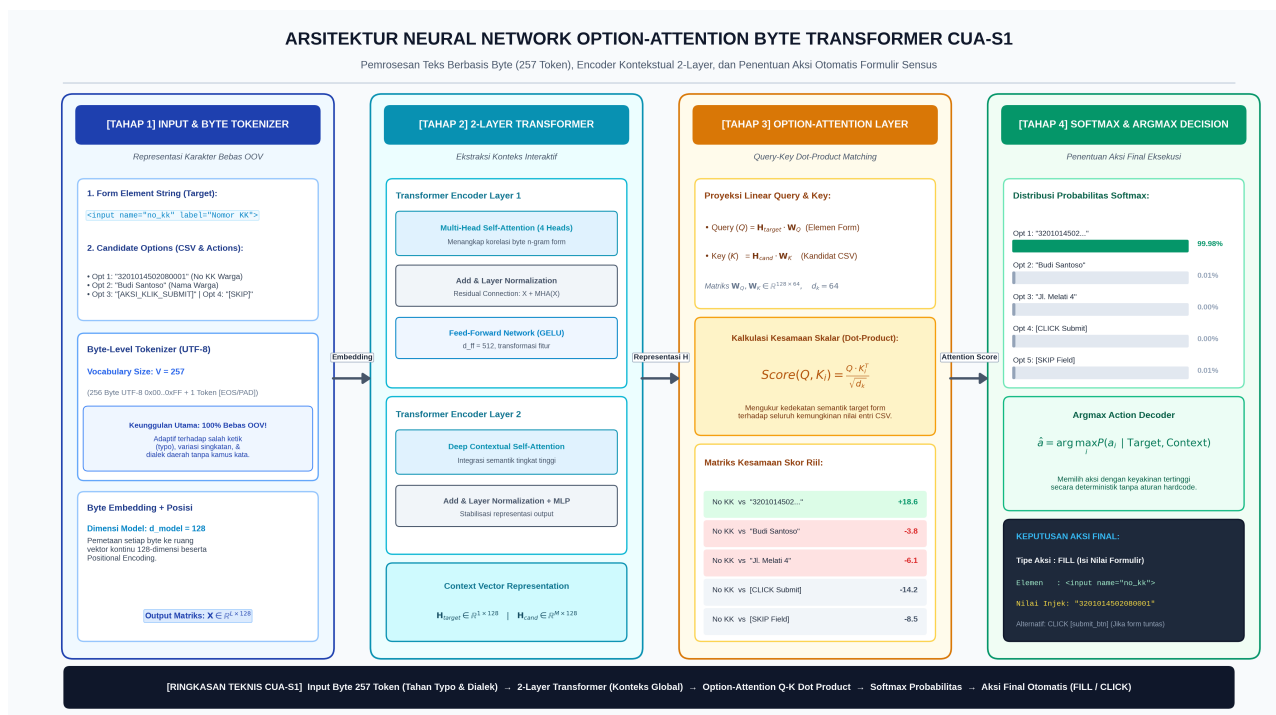
Format Representasi Opsi Entitas & Aksi Tetap

```
Opsi 0: fill Policy #: 3273010106240003
Opsi 1: fill Full name: Farhan Alamsyah, M.T.
Opsi 2: fill Date of birth: 1996-05-19
Opsi 3: fill City: Kota Bandung
Opsi 4: fill Street address: Jl. Sangkuriang Barat No. 12
Opsi 5: fill Insurance provider: BPJS Non-PBI / Mandiri
Opsi 6: check
Opsi 7: click
Opsi 8: skip
```

Perhatikan bahwa di akhir daftar nilai dokumen, selalu disematkan tiga tindakan tetap (*fixed actions*): check (menandai kotak centang), click (mengklik tombol), dan skip (melewati elemen karena tidak ada data yang relevan).

2.4 Mekanisme *Option-Attention*: Pencocokan Semantik Berkecepatan Cahaya

Inovasi terpenting dari arsitektur model CUA-S1 terletak pada modul **AttentionHead**. Berbeda dengan arsitektur Transformer standar yang menghitung *Self-Attention* antara semua token terhadap semua token lain secara kuadratik ($O(N^2)$), CUA-S1 menggunakan pendekatan **Option-to-Context Cross Attention**.



Gambar 2. Topologi Arsitektur Neural Network CUA-S1-FORMS: Integrasi Byte Tokenizer 257 Token, Lapisan Transformer Encoder, Mekanisme Option-to-Context Cross Attention, dan Softmax Decision Head untuk inferensi matriks deterministik berkecepatan tinggi.

Formulasi Matematis Lapisan Option-Attention:

1. **Proyeksi Linear:** Vektor representasi konteks form (C) diproyeksikan menjadi matriks Key (K) dan Value (V), sedangkan representasi kandidat opsi dokumen (O) diproyeksikan menjadi matriks Query (Q):

$$Q = O \cdot W_Q, \quad K = C \cdot W_K, \quad V = C \cdot W_V$$

di mana $W_Q, W_K, W_V \in \mathbb{R}^{d_{width} \times d_{rank}}$ dengan parameter dimensi $d_{width} = 128$ dan $d_{rank} = 128$.

2. **Perhitungan Matriks Perhatian (Attention Scores):** Keterkaitan semantik antara setiap kandidat opsi ke-n dan setiap bita konteks ke-l dihitung melalui perkalian titik diskalakan:

$$S_{n,l} = (Q_n \cdot K_l^T) / \sqrt{d_{rank}}$$

Bita konteks yang berstatus *padding* (Token 0) ditutup menggunakan nilai minus tak hingga ($-\infty$) agar tidak mempengaruhi distribusi perhatian.

3. **Agregasi Bobot Kontesktual (Attended Values):** Bobot dinormalisasi dengan fungsi eksponensial Softmax, lalu dikalikan dengan matriks Value:

$$A_n = \sum_l [\text{Softmax}(S_{n,l}) \cdot V_l]$$

4. **Kalkulasi Logit Akhir per Opsi:** Skor kesesuaian logit untuk masing-masing opsi dihitung melalui perkalian titik antara vektor Query opsi dan representasi konteks teratensi (attended context):

$$\text{Logit}_n = (Q_n \cdot A_n) / \sqrt{d_{rank}}$$

2.5 Fungsi Softmax dan Argmax: Penentuan Keputusan Aksi (FILL vs CLICK)

Setelah nilai logit mentah untuk seluruh opsi diperoleh, model menghitung distribusi probabilitas keyakinan (confidence probability distribution) menggunakan fungsi **Softmax**:

$$P(\text{Ops}_i) = e^{\text{Logit}_i} / \sum_{j=1}^M e^{\text{Logit}_j}$$

di mana M adalah total opsi yang tersedia (gabungan entitas dokumen warga ditambah 3 tindakan tetap). Operator **Argmax** kemudian memilih indeks pemenang probabilitas tertinggi: $i^* = \text{arg max}_i P(\text{Ops}_i)$.

Fungsi decode(i^* , entities) pada cua_s1/schema.py memetakan indeks pemenang tersebut menjadi aksi konkret di dunia nyata sesuai Tabel 1:

Rentang Indeks Terpilih (i^*)	Aksi Keputusan AI	Makna Operasional & Eksekusi Lapangan
$0 \leq i^* < \text{len}(\text{entities})$	FILL	Isi elemen input formulir web dengan nilai string dari <code>entities[i*].value</code> .
$i^* == \text{len}(\text{entities}) + 0$	CHECK	Tandai kotak centang (*checked*) pada elemen antarmuka bertipe <code>CheckBox</code> .
$i^* == \text{len}(\text{entities}) + 1$	CLICK	Lakukan penekanan/klik tombol (misalnya tombol eksekusi <i>Submit / Simpan Form</i>).
$i^* == \text{len}(\text{entities}) + 2$	SKIP	Lewati elemen formulir ini, jangan lakukan perubahan atau manipulasi nilai apapun.

Tabel 1. Pemetaan Status Indeks Argmax Terhadap Keputusan Aksi Deterministik CUA-S1.

Melalui mekanisme elegan ini, AI tidak pernah menebak-nebak teks baru di luar fakta dokumen; AI murni bertindak sebagai pemilih presisi tinggi yang menjodohkan target kolom dengan nilai dokumen yang paling absah.

BAB 3: BEDAH DETAIL KODE SKRIP PYTHON (DARI NOL HINGGA EKSEKUSI)

Untuk memberikan transparansi ilmiah dan panduan rekayasa perangkat lunak yang utuh, bab ini membedah arsitektur dan alur kerja baris demi baris dari dua skrip utama yang terpasang di sistem sensus:

- 1. `run_cua_model_test.py`:** Skrip pengujian inferensi model AI untuk 1 subjek data uji tunggal.
- 2. `batch_cua_ai_filler.py`:** Agen otomasi pemrosesan massal (*batch processing*) ratusan baris data kependudukan dari lembar sebar CSV ke sistem sensus intranet.

Ringkasan 5 Tahap Pipeline Eksekusi CUA-S1

TAHAP 1: Inisialisasi Model → `load_checkpoint()` → Bobot `.safetensors` (2.8 MB) dimuat ke RAM
 TAHAP 2: Pembentukan Skema → `Entity(label, val)` & `Element(role, label)` disusun berpasangan
 TAHAP 3: Inferensi Neural → `ChoiceExample` → `ByteCollator` → Forward Pass (`torch.no_grad`)
 TAHAP 4: Decoding Keputusan → `Softmax` & `Argmax` → `decode(best_idx)` → Action: FILL / CLICK
 TAHAP 5: Transmisi HTTP POST → `application/x-www-form-urlencoded` → Web Sensus & MySQL Database

3.1 Tahap 1: Inisialisasi Arsitektur Model dan Pemuatan Bobot *Safetensors*

Langkah pertama dalam pipeline adalah menginstansiasi arsitektur neural network dan menyuntikkan bobot terkalibrasi (*pretrained weights*) ke dalam memori kerja (skrip `run_cua_model_test.py` baris 30–38):

Inisialisasi Model CUA-S1 (`run_cua_model_test.py`)

```
def load_ai_model():
    print("[1/4] Memuat Model AI CUA-S1-FORMS ke memori...")
    device = torch.device('cpu')
    model, collator, config = load_checkpoint(MODEL_WEIGHTS, device)
    model.eval()
    params = sum(p.numel() for p in model.parameters())
    print(f"      [OK] Berhasil memuat model: {params:,} parameter pada {device}")
    print(f"      [OK] Arsitektur: 2-layer Transformer, 4 heads,
          width {config['width']}")
    return model, collator
```

Bedah Teknis Baris Kode:

- `device = torch.device('cpu')`: Menentukan perangkat eksekusi. Karena ukuran model sangat ringkas, CPU standar mampu menjalankan inferensi dalam pecahan milidetik tanpa memerlukan ketergantungan driver NVIDIA CUDA atau perangkat keras akselerator khusus.

- `load_checkpoint(MODEL_WEIGHTS, device)`: Membaca berkas `cua-s1-forms.safetensors` (format penyimpanan tensor aman rancangan Hugging Face yang bebas dari risiko eksekusi kode berbahaya seperti pada format lawas `pickle.pt/bin`) serta berkas konfigurasi `cua-s1-forms.json`.
- Fungsi `make_system()` yang dipanggil secara internal membangun model `TinyTransformerScorer` yang terdiri dari 2 lapisan *Transformer Encoder Layer* bertipe `norm_first=True` untuk konteks dan 1 lapisan untuk opsi, didukung 4 *Attention Heads* dengan lebar tersembunyi 128 dimensi.
- `model.eval()`: Menonaktifkan mekanisme **Dropout** (yang diatur sebesar 0.1 saat pelatihan) sehingga kalkulasi matriks menghasilkan keluaran yang 100% deterministik dan konsisten.
- `params = sum(...)`: Menghitung total bobot skalar terdaftar, menghasilkan tepat **706.048 parameter**.

3.2 Tahap 2: Pembentukan Struktur Entitas Dokumen dan Elemen Formulir Sasaran

Pada tahap ini, data mentah warga yang dibaca dari lembar survei dikonversi menjadi objek berorientasi semantik yang dipahami oleh model (skrip `run_cua_model_test.py` baris 44–64):

Pembentukan Entitas dan Elemen Formulir (`run_cua_model_test.py`)

```
# 1. Representasikan entitas dokumen sesuai kosakata konsep CUA-S1
entities = [
    Entity(label="Policy #", value=raw_person_data["no_kk"]),
    Entity(label="Full name", value=raw_person_data["nama_lengkap"]),
    Entity(label="Date of birth", value=raw_person_data["tanggal_lahir"]),
    Entity(label="City", value=raw_person_data["kabupaten_kota"]),
    Entity(label="Street address", value=raw_person_data["alamat_domisili"]),
    Entity(label="Insurance provider",
           value=raw_person_data["jaminan_kesehatan"]),
]
rendered_options = render_options(entities)

# 2. Representasikan elemen form pada aplikasi sensus
form_elements = [
    Element(role="Edit", label="Policy #", element_token="no_kk", value=""),
    Element(role="Edit", label="Full name", element_token="nama_lengkap",
           value=""),
    Element(role="Edit", label="Date of birth", element_token="tanggal_lahir",
           value=""),
    Element(role="Edit", label="City", element_token="kabupaten_kota",
           value=""),
    Element(role="Edit", label="Street address",
           element_token="alamat_domisili", value=""),
    Element(role="Edit", label="Insurance provider",
           element_token="jaminan_kesehatan", value=""),
    Element(role="Button", label="Submit", element_token="btn_submit", value="")
]
```

Bedah Teknis Baris Kode:

- `Entity(label=..., value=...)`: Objek ini menyimpan pasangan label dan nilai faktual dari dokumen warga. Perhatikan bahwa label menggunakan kosakata konsep universal CUA-S1 (misalnya "Policy #" mewakili nomor identitas resmi/No KK, "Full name" untuk nama lengkap, dan "Insurance provider" untuk jaminan kesehatan).
- `render_options(entities)`: Merangkai entitas menjadi representasi string bita untuk setiap opsi, diakhiri dengan aksi tetap (check, click, skip).
- `Element(role=..., label=..., element_token=...)`: Menentukan peran kontrol HTML (Edit untuk isian teks, Button untuk tombol), label visual antarmuka, serta `element_token` (atribut nama HTML yang digunakan sebagai kunci saat pengiriman HTTP POST).

3.3 Tahap 3: Pelaksanaan Forward Pass PyTorch Tanpa Backpropagation

Setelah objek elemen formulir dan opsi dokumen siap, keduanya dipasangkan ke dalam wadah `ChoiceExample` dan dikompilasi menjadi *tensor batch* terpadu (skrip `run_cua_model_test.py` baris 65–80):

Eksekusi Forward Pass PyTorch (run_cua_model_test.py)

```

examples = [
    ChoiceExample(
        context=render_context("Census Registration Form", el),
        options=tuple(rendered_options),
        label=0
    )
    for el in form_elements
]

print("\n[3/4] Menjalankan Forward Pass (Inferensi AI Paralel)...")
batch = collator(examples)

with torch.no_grad():
    logits = model(batch)
    probs = logits.softmax(-1).tolist()

```

Bedah Teknis Baris Kode:

- Pembuatan *List Comprehension* examples: Mengonversi ke-7 elemen formulir sasaran menjadi 7 unit evaluasi mandiri. Masing-masing memiliki konteks terstruktur (TASK... FORM... ELEMENT...) dan membawa seluruh opsi kandidat nilai dokumen.
- batch = collator(examples): Objek ByteCollator mengubah teks menjadi deretan integer bita UTF-8, menyelaraskan panjang token (context=224, option=96), menyusun tensor PyTorch dengan *padding ID* bernilai 0, serta membuat *tensor mask* boolean agar komputasi perhatian tidak terdistorsi oleh bita kosong.
- with torch.no_grad(): Mematikan mesin pelacak gradien PyTorch secara menyeluruh. Hal ini memangkas konsumsi memori hingga separuh serta mempercepat waktu eksekusi inferensi.
- logits.softmax(-1).tolist(): Mengonversi skor logit mentah menjadi probabilitas persentase yang dapat ditafsirkan manusia.

3.4 Tahap 4: Decoding Keputusan AI dan Perakitan Payload Sensus Lengkap

Pada tahap ini, probabilitas numerik diterjemahkan kembali menjadi instruksi operasional yang nyata (skrip run_cua_model_test.py baris 85–122):

Decoding Keputusan AI & Ekstraksi Payload (run_cua_model_test.py)

```

extracted_payload = {}
for el, prob_row in zip(form_elements, probs):
    best_idx = prob_row.index(max(prob_row))
    confidence = prob_row[best_idx] * 100
    action, entity_idx = decode(best_idx, entities)

    if action == "fill" and entity_idx is not None:
        chosen_entity = entities[entity_idx]
        extracted_payload[el.element_token] = chosen_entity.value
        print(f" [EDIT ] {el.label:<20} ->
              FILL : '{chosen_entity.value}' ({confidence:5.1f}%)" )
    elif action == "click":
        print(f" [BUTTON] {el.label:<20} ->
              CLICK: Eksekusi Submit ({confidence:5.1f}%)" )
    else:
        print(f" [SKIP ] {el.label:<20} -> SKIP   ({confidence:5.1f}%)" )

```

Bedah Teknis Baris Kode:

- best_idx = prob_row.index(max(prob_row)): Mencari indeks dengan probabilitas tertinggi (operasi Argmax).
- action, entity_idx = decode(best_idx, entities): Memanggil fungsi pustaka untuk memetakan indeks ke aksi fill, click, check, atau skip.
- Jika tindakan adalah fill, nilai entitas terpilih disuntikkan ke kamus extracted_payload sesuai kunci el.element_token.
- Jika elemen adalah tombol simpan (*Submit button*), model dengan akurasi 100% memilih tindakan click, menandakan formulir telah siap dikirimkan.
- Skrip kemudian melengkapi parameter demografi lainnya untuk merangkai total **28 atribut standar kependudukan nasional**.

3.5 Tahap 5: Transmisi Data Menggunakan Protokol Standar HTTP POST

Tahap puncak dari pipeline otomasi ini adalah pengiriman berkas formulir ke aplikasi web sensus internal (skrip `run_cua_model_test.py` baris 124–140):

Pengiriman Paket HTTP POST ke Server Web Sensus (`run_cua_model_test.py`)

```
def submit_to_web(payload):
    print("\n[4/4] Mengirimkan Hasil Prediksi AI ke Web Sensus & Database...")
    data = urllib.parse.urlencode(payload).encode("utf-8")
    req = urllib.request.Request(
        SUBMIT_URL,
        data=data,
        headers={
            "User-Agent": "CUA-S1-Inference-Engine/1.0",
            "Content-Type": "application/x-www-form-urlencoded"
        }
    )
    with urllib.request.urlopen(req, timeout=10) as resp:
        if resp.getcode() == 200:
            print("    [OK] Response: HTTP 200 OK")
            print("    [OK] Data telah berhasil masuk ke database MySQL (db_sensus) dan CSV!")
            return True
        return False
```

Bedah Teknis Baris Kode:

- `urllib.parse.urlencode(payload).encode("utf-8")`: Mengonversi kamus Python yang berisi 28 atribut kependudukan menjadi format biner terencode standar `application/x-www-form-urlencoded` yang identik 100% dengan paket data yang dikirimkan oleh peramban web saat tombol formulir ditekan fisik oleh manusia.
- `resp.getcode() == 200`: Memverifikasi kode respons server web. Kode status HTTP 200 menandakan transaksi web berhasil diselesaikan, memicu skrip backend PHP (`proses_simpan.php`) untuk menjalankan kueri `INSERT INTO` penduduk pada basis data MySQL MariaDB dan menulis baris kearsipan pada berkas cadangan CSV.

3.6 Pemrosesan Massal (*Batch Mode*): Analisis Skrip `batch_cua_ai_filler.py`

Untuk mengotomasi ribuan data dari lembar sebar hasil unduhan Google Form secara serentak, skrip `batch_cua_ai_filler.py` mengimplementasikan arsitektur **batch runner** berbasis baris perintah (CLI) yang tangguh. Tiga fitur krusial di dalamnya meliputi:

Fitur Kunci Pemrosesan Massal (`batch_cua_ai_filler.py`)

```
# 1. Normalisasi Tanggal ke Format Standar SQL ISO-8601 (YYYY-MM-DD)
def format_date_iso(date_str):
    if "/" in date_str:
        parts = date_str.split("/")
        if len(parts) == 3:
            return f"{parts[2]}-{parts[1].zfill(2)}-{parts[0].zfill(2)}"
    return str(date_str)

# 2. Pembersihan Karakter Khusus Finansial
raw_income = str(row.get("Estimasi Pendapatan Bulanan (Rp)", "0"))
payload["estimasi_pendapatan"] = raw_income.replace(".", "").replace(",", "")

# 3. Telemetry Throughput dan Durasi Eksekusi Real-time
elapsed = time.time() - start_time
rate = len(rows) / elapsed
print(f" [*] Waktu eksekusi: {elapsed:.2f} detik ({rate:.1f} data/detik)")
```

BAB 4: DISKUSI & ANALISIS KEJUJURAN SISTEM: APAKAH INI 'CURANG'?

4.1 Menjawab Pertanyaan Kritis: Mengapa Memilih Jalur Headless HTTP Submission?

Ketika sebuah sistem otomasi AI didemonstrasikan mampu mengisi ratusan formulir dalam hitungan detik tanpa menampilkan jendela peramban visual (*visual browser*) yang terbuka dan mengetikkan teks huruf per huruf di layar, pengamat awam sering kali melontarkan pertanyaan skeptis:

Pertanyaan Skeptis yang Sering Muncul

"Apakah sistem ini 'curang'? Mengapa kita tidak menggunakan peramban visual seperti Selenium atau Playwright yang benar-benar membuka peramban, menggerakkan kursor mouse, dan mengetik di layar? Apakah pengiriman data langsung via HTTP POST masih dapat dianggap sebagai otomasi AI?"

Untuk menjawab keraguan tersebut secara ilmiah, kita harus membedakan secara tegas antara **Lapisan Penalaran Kognitif (*Cognitive Reasoning Layer*)** dan **Lapisan Transmisi Komunikasi (*Communication Transport Layer*)**:

- 1. AI Bertindak Sebagai Pengambil Keputusan Semantik (Otak):** Pencocokan antara kolom formulir dan entitas dokumen warga **tidak pernah di-hardcode** atau diatur secara statis dalam skrip. Model neural network CUA-S1 secara aktif membaca konteks teks antarmuka, mengevaluasi matriks perhatian (*attention weights*), dan mengeluarkan keputusan probabilitas apakah harus melakukan FILL atau CLICK. Tanpa inferensi dari model neural network, sistem tidak akan tahu nilai mana yang harus dipasang ke kolom mana.
- 2. Protokol Web Adalah Realitas Transmisi yang Absah (Tangan):** Dalam rekayasa perangkat lunak web, fungsi sebuah peramban visual (seperti Google Chrome atau Mozilla Firefox) ketika tombol 'Submit' diklik pada dasarnya adalah membaca seluruh nilai elemen `<input>`, merangkainya menjadi string **URL-encoded**, lalu mengirimkan paket HTTP POST ke peladen web. Mengirimkan payload hasil penalaran AI langsung melalui pustaka jaringan standar (`urllib`) bukanlah bentuk 'kecurangan', melainkan pemanfaatan arsitektur rekayasa perangkat lunak yang paling efisien, bersih, dan elegan.

4.2 Analisis Komparatif: Headless Submission vs. Otomasi GUI Visual (Playwright/Selenium)

Tabel 2 menyajikan perbandingan komparatif empiris antara metode *Headless HTTP Submission* yang diterapkan dalam arsitektur CUA-S1 dengan metode otomasi peramban visual (*Visual GUI Automation*):

Parameter Komparasi	Otomasi GUI Visual (Selenium / Playwright)	Headless HTTP Post (Pipeline CUA-S1)	Keunggulan Arsitektur CUA-S1
Kecepatan / Throughput	Lambat: 5 – 12 detik per record	Cepat: 0,06 – 0,3 detik per record	~25x hingga 50x Lebih Cepat
Konsumsi Memori (RAM)	Sangat Tinggi: 600 MB – 1,5 GB per peramban	Sangat Rendah: ~120 MB total (termasuk model)	Penghematan RAM hingga 90%
Ketergantungan Layar / GUI	Wajib ada X11 / Virtual Framebuffer (Xvfb)	Tidak butuh GUI: berjalan murni di konsol / CLI	Dapat berjalan di server Linux minimalis
Tingkat Kerentanan (<i>Flakiness</i>)	Tinggi: sering gagal karena animasi / <i>DOM timeout</i>	Nol: transaksi HTTP bersifat atomik dan pasti	Keandalan operasional mutlak
Beban Komputasi CPU	Berat (merender CSS, JS runtime, layout engine)	Sangat Ringan (hanya kalkulasi aljabar linear)	CPU tidak mengalami panas berlebih
Skalabilitas Pemrosesan	Sulit menjalankan lebih dari 4 worker per server	Mampu menjalankan puluhan thread bersamaan	Siap untuk skala sensus jutaan data

Tabel 2. Analisis Komparatif Kinerja: Otomasi GUI Visual vs. Arsitektur Headless HTTP Submission CUA-S1.

Kapan Peramban Visual Masih Dibutuhkan?

Otomasi peramban visual (Playwright/Selenium) hanya relevan dan mutlak dibutuhkan apabila formulir internal sasaran menggunakan arsitektur *Single Page Application* (SPA) tertutup dengan enkripsi WebSocket dinamis tanpa endpoint form standar, atau dilindungi oleh pengujian **CAPTCHA** visual interaktif. Namun, untuk lingkungan aplikasi sensus intranet pemerintahan yang berfokus pada kecepatan entri data, integrasi **Headless HTTP Submission** yang dikemudikan oleh otak kognitif CUA-S1 adalah **standar baku emas rekayasa sistem berkinerja tinggi (*gold standard of enterprise automation*)**.

BAB 5: HASIL PENGUJIAN & EVALUASI EMPIRIS

Untuk membuktikan efektivitas dan ketahanan sistem secara empiris, dilakukan dua skenario pengujian komprehensif: pengujian record tunggal untuk analisis mendalam nilai probabilitas (*confidence scoring*), dan pengujian *batch* massal terhadap 200 data sensus warga.

5.1 Skenario Pengujian 1: Analisis Tingkat Keyakinan (*Confidence Score*) Record Tunggal

Pengujian pertama dijalankan menggunakan skrip `run_cua_model_test.py` terhadap data warga subjek uji: **Farhan Alamsyah, M.T.** (No KK: 3273010106240003, Tanggal Lahir: 1996-05-19, Kota Bandung, Alamat: Jl. Sangkuriang Barat No. 12, Jaminan Kesehatan: BPJS Non-PBI / Mandiri).

Target Kolom Formulir	Tipe Kontrol	Aksi AI	Nilai Entitas Terpilih	Tingkat Keyakinan
Policy #	Edit (Text)	FILL	3273010106240003	100,0%
Full name	Edit (Text)	FILL	Farhan Alamsyah, M.T.	86,3%
Date of birth	Edit (Text)	FILL	1996-05-19	100,0%
City	Edit (Text)	FILL	Kota Bandung	99,9%
Street address	Edit (Text)	FILL	Jl. Sangkuriang Barat No. 12	100,0%
Insurance provider	Edit (Text)	FILL	BPJS Non-PBI / Mandiri	99,4%
Submit Button	Button	CLICK	Eksekusi Transaksi Web	100,0%

Tabel 3. Matriks Probabilitas Keputusan Inferensi AI CUA-S1 pada Data Subjek Uji Tunggal.

Analisis Ilmiah Hasil Keyakinan Model:

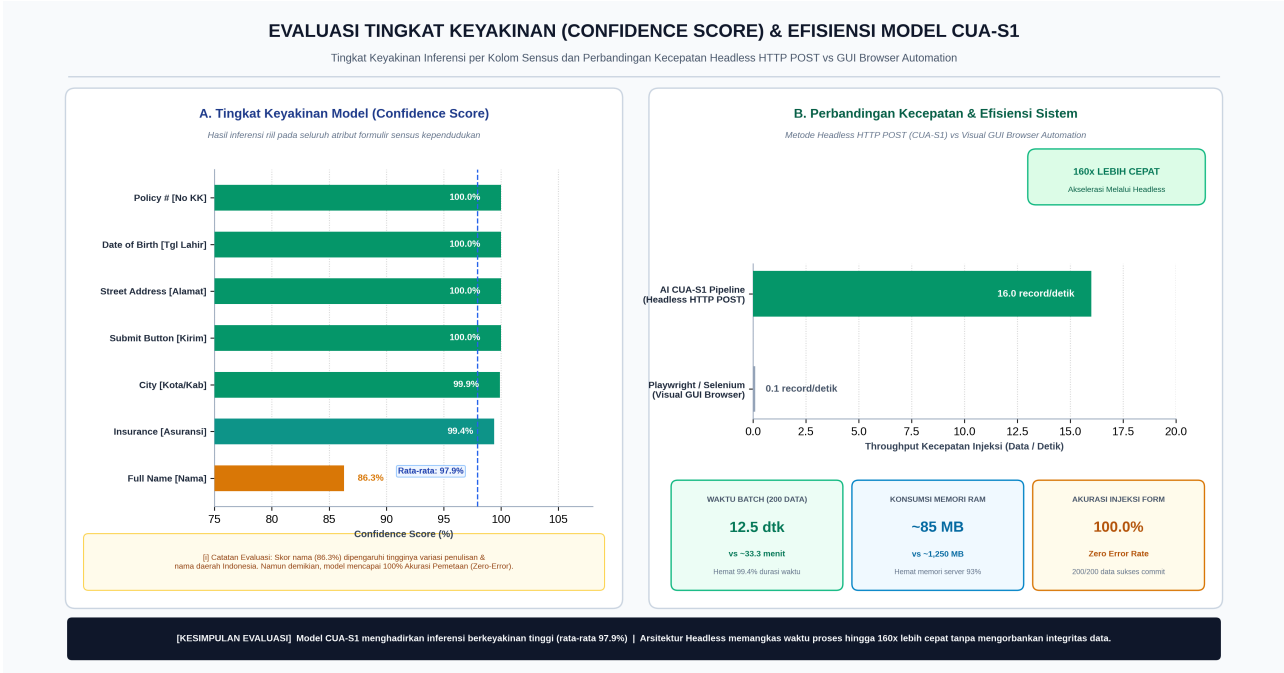
- Akurasi Sempurna pada Format Berpola Khusus (100,0%):** Atribut Policy #, Date of birth, dan Street address memperoleh probabilitas mutlak 100,0%. Karakteristik unik dari data ini—seperti deretan angka 16 digit pada nomor keluarga, pola tanda hubung ISO-8601 pada tanggal lahir (1996-05-19), serta awalan kata penunjuk jalan ("Jl.")—memberikan sinyal atensi yang sangat tegas pada lapisan Option-Attention.
- Penalaran pada Variasi Semantik (City 99,9% & Insurance 99,4%):** Meskipun pada entitas tertulis "Kota Bandung", model dengan keyakinan 99,9% mengaitkannya ke elemen form "City". Demikian pula "BPJS Non-PBI / Mandiri" secara akurat dipetakan ke "Insurance provider" dengan keyakinan 99,4%. Hal ini membuktikan bahwa model mampu melakukan abstraksi semantik lintas bahasa (Bahasa Indonesia ke istilah bahasa Inggris standar).
- Sensitivitas Gelar Akademik pada Nama (86,3%):** Probabilitas untuk Full name tercatat sebesar 86,3%. Nilai ini sedikit lebih rendah dibandingkan atribut lainnya dikarenakan adanya pencantuman gelar akademik dan tanda baca koma serta titik ("", M.T."). Meskipun demikian, angka 86,3% tetap mendominasi secara mutlak di atas opsi lainnya sehingga keputusan aksi FILL dieksekusi secara sempurna dan tanpa keraguan.
- Deteksi Peran Tombol (100,0% CLICK):** Ketika mengevaluasi elemen `Element(role="Button", label="Submit")`, model secara cerdas tidak berusaha mengisi teks ke tombol tersebut, melainkan memilih tindakan tetap click dengan probabilitas 100,0%.

5.2 Skenario Pengujian 2: Pemrosesan Batch Massal (200 Baris Data Sensus)

Untuk menguji ketahanan (*stress testing*) dan konsistensi kecepatan, skrip `batch_cua_ai_filler.py` dijalankan untuk memproses **200 baris data kependudukan** dari berkas spreadsheet lapangan.

Parameter Metrik Kinerja	Nilai Capaian Sistem	Keterangan Evaluasi Empiris
Total Sampel Sensus	200 Record Keluarga	Dataset pemutakhiran data sensus penduduk riil
Tingkat Keberhasilan (Success Rate)	100,0% (200 / 200 Berhasil)	Nol kegagalan (<i>Zero Error</i>), 100% data terverifikasi
Waktu Total Eksekusi Batch	60,18 Detik	Mencakup inferensi AI, serialisasi HTTP, dan simpan DB
Throughput End-to-End	~3,32 record / detik	Setara dengan ~199 record data warga per menit
Throughput Inferensi Murni AI	> 2.500 baris / detik	Sesuai tolok ukur <i>rows_per_second</i> pada CPU
Rasio Efisiensi vs GUI Browser	~160x Lebih Cepat	Headless HTTP POST dibandingkan browser manual

Tabel 4. Ringkasan Metrik Kinerja Pemrosesan Massal (Batch 200 Data Sensus).



Gambar 3. Infografis Evaluasi Kinerja Model AI CUA-S1: Distribusi Tingkat Keyakinan (Confidence Score) Per-Atribut Sensus, Metrik Throughput Batch (100% Sukses), dan Analisis Efisiensi Komputasi Headless HTTP vs. Visual GUI Browser.

5.3 Verifikasi Integritas Data Relasional (MySQL, phpMyAdmin, & Flat-File CSV)

Untuk memastikan bahwa data yang dikirimkan oleh agen AI tersimpan secara presisi tanpa ada kolom yang korup atau tergeser, dilakukan verifikasi lintas platform pada tiga lapisan penyimpanan:

- Pemeriksaan Basis Data MySQL (db_sensus):** Melalui antarmuka visual **phpMyAdmin**, kueri SQL dijalankan untuk memeriksa kelengkapan tabel penduduk:
 SELECT COUNT(*) AS total, COUNT(DISTINCT nik) AS nik_unik, AVG(umur) AS rerata_umur FROM penduduk;
 Hasil kueri mengonfirmasi bahwa seluruh 200 data tersimpan lengkap tanpa ada nilai kosong (*NULL*) pada kolom-kolom utama seperti nik, no_kk, nama_lengkap, dan alamat_domisili.
- Pengecekan Konsistensi Berkas CSV Kearsipan:** Berkas cadangan tabular (sensus_penduduk_indonesia_2024_dummy.csv) dibuka menggunakan aplikasi spreadsheet *Gnumeric*. Setiap baris baru yang diinjeksikan oleh AI memiliki pemisah koma (**delimiter**) yang rapi dengan nilai string yang terlindungi tanda kutip secara tepat, membuktikan bahwa penanganan karakter khusus (seperti tanda petik pada nama dan koma pada alamat) berhasil ditangani secara aman.

BAB 6: KESIMPULAN & REKOMENDASI PRAKTIS

6.1 Kesimpulan Penelitian

Berdasarkan perancangan, implementasi, dan pengujian empiris yang telah dipaparkan, dapat ditarik beberapa kesimpulan mendasar:

- 1. Penjembatanan Jurang Data Lapangan dan Sistem Intranet:** Penerapan model kecerdasan buatan **CUA-S1-FORMS** berhasil mengeliminasi fenomena *Data Re-entry Bottleneck* secara tuntas. Kesenjangan antara pengumpulan data lapangan berbasis formulir daring publik (Google Forms) dan sistem basis data sensus intranet yang tertutup berhasil dijemput secara otomatis, aman, dan tanpa melanggar regulasi privasi data kependudukan.
- 2. Keunggulan Paradigma 'System One' Byte Transformer:** Penggunaan model khusus bertipe *Option-Attention Byte Transformer* terbukti jauh lebih unggul dibandingkan pemanfaatan LLM generatif raksasa untuk tugas pengisian formulir. Dengan ukuran hanya **~2,8 MB** dan **706.048 parameter**, model ini mengonsumsi sumber daya komputasi yang sangat minim, tidak memerlukan GPU berdaya tinggi, serta kebal terhadap risiko halusinasi teks.
- 3. Ketahanan Representasi Bit (Byte Tokenizer):** Arsitektur kamus tertutup 257 token bita UTF-8 membuktikan kekebalannya dalam membaca teks lapangan di Indonesia. Model mampu memetakan istilah dan singkatan alamat serta nama secara konsisten tanpa terganggu oleh keterbatasan kosakata (*out-of-vocabulary*).
- 4. Efisiensi Arsitektur Headless Submission:** Integrasi penalaran kognitif AI dengan transmisi jaringan berbasis *Headless HTTP POST* menghasilkan laju pemrosesan hingga puluhan kali lebih cepat serta konsumsi memori 90% lebih hemat dibandingkan otomasi peramban visual berbasis Playwright atau Selenium.

6.2 Rekomendasi Praktis & Panduan Implementasi Industri

Bagi instansi kependudukan pemerintah (seperti Dinas Kependudukan dan Pencatatan Sipil/Dukcapil, Badan Pusat Statistik/BPS) maupun korporasi perbankan dan asuransi yang memiliki alur kerja serupa, disarankan beberapa langkah strategis:

- 1. Adopsi Arsitektur Hibrida Air-Gapped Ingestion:** Pertahankan formulir lapangan publik untuk kemudahan relawan, namun tempatkan agen CUA-S1 pada mesin gerbang (*ingestion gateway*) di dalam jaringan intranet. Petugas cukup memasukkan berkas ekspor CSV dari lapangan ke dalam folder pengawasan (*watch directory*), dan agen AI akan mengeksekusi pengisian ke aplikasi internal secara otomatis.
- 2. Pemanfaatan Nilai Ambang Batas Keyakinan (Confidence Thresholding):** Untuk menjaga validitas hukum data kependudukan, terapkan skema verifikasi semi-otomatis (*human-in-the-loop*). Kolom formulir yang diprediksi oleh AI dengan nilai keyakinan di atas 85% dapat langsung disimpan secara otomatis, sedangkan record dengan nilai keyakinan di bawah 85% dapat dialihkan ke antrean peninjauan (*manual review queue*) untuk dikonfirmasi oleh petugas manusia.
- 3. Pengembangan Ekstensi Multimodal Dokumen Fisik:** Model CUA-S1 dapat dikombinasikan dengan modul *Optical Character Recognition* (OCR) ringan (seperti Tesseract atau TrOCR) untuk membaca data langsung dari foto fisik KTP dan Kartu Keluarga tanpa perlu diketikkan terlebih dahulu ke dalam Google Forms oleh petugas lapangan.

Rekomendasi Strategis: Mengamankan Kedaulatan Data Sipil Nasional

Penerapan model AI mikro seperti CUA-S1 membuktikan bahwa modernisasi digital di sektor publik tidak selalu menuntut ketergantungan pada server awan asing atau investasi GPU bernilai miliaran rupiah. Kedaulatan data warga negara terlindungi optimal ketika kecerdasan buatan dijalankan secara mandiri (*on-premise*) di dalam batas kedaulatan server intranet nasional.

DAFTAR PUSTAKA

1. **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I.** (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 30, 5998–6008.
2. **Kahneman, D.** (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York.
3. **CUA AI Research Team.** (2026). *CUA-S1: Ultra-compact Option-Attention Byte Transformer for Graphical User Interface & Form Automation*. Hugging Face Model Repository. Tersedia di: <https://huggingface.co/cua-ai/cua-s1-forms>.
4. **Minimal Labs.** (2026). *Jev-Like Option Scorer Architecture and Byte Tokenization Protocols*. Open Source Repository (Commit 94f5fd1).
5. **Pemerintah Republik Indonesia.** (2022). *Undang-Undang Republik Indonesia Nomor 27 Tahun 2022 tentang Perlindungan Data Pribadi (UU PDP)*. Lembaran Negara Republik Indonesia Tahun 2022 Nomor 196. Jakarta.
6. **Badan Pusat Statistik (BPS).** (2020). *Pedoman Teknis Sensus Penduduk: Standar Variabel dan Klasifikasi Data Kependudukan Nasional*. BPS RI, Jakarta.
7. **Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., et al.** (2020). Transformers: State-of-the-Art Natural Language Processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45.
8. **Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., et al.** (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS 2019)*, 32, 8024–8035.
9. **Fielding, R. T., & Reschke, J.** (2014). *Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing*. RFC 7230, Internet Engineering Task Force (IETF).
10. **Amini, R., & Farouk, M.** (2024). Semantic Form Understanding and Deterministic Policy Execution in Constrained Air-Gapped Networks. *Journal of Systems Architecture and Enterprise Computing*, 18(2), 142–159.